Docker MCP Gateway - CVE-2026-55887

Argument Injection Through OCI Image Label Metadata

Language: English
Date: 2026-06-29
Author: Security Lab
Classification: Defensive security research and local laboratory documentation

Contents

Abstract

CVE-2026-55887 is a high-severity argument-injection vulnerability in Docker MCP Gateway / Docker MCP Plugin. The vulnerable behavior affects versions from `0.21.0` up to, but not including, `0.42.2`. The flaw is triggered through attacker-controlled metadata stored in the `io.docker.server.metadata` OCI image label. In vulnerable versions, that label is parsed into a broad server configuration structure that includes runtime-shaping fields, allowing untrusted image metadata to influence the `docker run` arguments used to launch an MCP server container.

This report documents the vulnerability, the technical root cause, the local laboratory architecture, the exploitation chain reproduced inside the lab boundary, the upstream remediation in `0.42.2`, and the defensive indicators that make the behavior observable.

Scope

The material is limited to defensive security research and local reproduction. The lab uses an isolated Docker-in-Docker victim daemon and does not require publishing malicious images to a public registry or exposing the real host Docker socket to the vulnerable gateway.

Key Facts

CVE	`CVE-2026-55887`
GitHub Advisory	`GHSA-r2xf-7jw5-pjg6`
ZDI Advisory	`ZDI-26-363`, `ZDI-CAN-29539`
Affected package	`github.com/docker/mcp-gateway`
Affected versions	`>= 0.21.0, < 0.42.2`
Fixed version	`0.42.2`
Weakness	`CWE-88`: argument delimiter neutralization failure
Reported severity	High
Reported scores	GitHub CVSS v4.0 `8.7`; GitLab CVSS v3.1 `8.6`; ZDI CVSS v3.0 `8.6`

Table 1: Consolidated vulnerability facts from public advisories.

Technical Root Cause

Docker MCP Gateway supports MCP server definitions carried through OCI image metadata. In vulnerable versions, the gateway parses the `io.docker.server.metadata` label as YAML into a broad server configuration structure. That structure contains descriptive fields, but it also contains fields that shape container runtime behavior.

The resulting issue is a mass-assignment style argument-injection flaw. Metadata controlled by the image publisher reaches fields reserved for trusted gateway configuration. Those fields then influence the Docker argument vector constructed by the gateway.

```
attacker-controlled OCI image
        ↓
io.docker.server.metadata label
        ↓
YAML parsed by vulnerable gateway
        ↓
runtime fields enter server configuration
        ↓
gateway builds docker run arguments
        ↓
container starts with attacker-influenced runtime options
```

The vulnerability is argument injection rather than classic shell injection. The dangerous values are inserted into Docker's argument vector rather than passed through a shell command string.

Impact

Public advisories describe impact paths where attacker-controlled metadata causes the gateway to start an MCP server container with dangerous Docker runtime options. Examples include host filesystem mounts, execution as UID `0`, and exposure of the Docker socket to the launched container.

The trust boundary is bypassed at container creation time. Container hardening flags such as `no-new-privileges` do not neutralize this issue because the dangerous options are applied before code inside the container runs.

In a real environment, successful exploitation can lead to host compromise. In this lab, the host role is intentionally replaced by a disposable Docker-in-Docker victim daemon.

Laboratory Architecture

The technical architecture of the lab is carried by the `lab/` directory. That directory separates the responsibilities of the environment: lifecycle control, victim Docker daemon, local registry, real gateway builds, MCP images, observation, and attacker workstation. Docker Compose then assembles these directories into runtime services.

```
lab/
|-- scripts/   # lifecycle, scenarios, evidence, and reset
|-- victim/    # Docker-in-Docker daemon representing the victim host
|-- registry/  # local OCI registry local-registry:5000
|-- gateway/   # real docker/mcp-gateway builds v0.42.1 and v0.42.2
|-- images/    # benign and challenge MCP images
|-- observer/  # event, gateway log, and proof collection
|-- attacker/  # local image build and inspection workstation
```

The `lab/gateway/` directory is central: it does not contain an imitation, but the definitions used to build and run the real upstream Docker MCP Gateway. The vulnerable path uses `v0.42.1`; the fixed path uses `v0.42.2`.

At runtime, `lab/victim/` becomes the `victim-docker` service, `lab/registry/` becomes `local-registry`, `lab/gateway/` produces `vulnerable-gateway` and `fixed-gateway`, `lab/observer/` becomes `observer`, and `lab/attacker/` becomes `attacker-workstation`. Images defined in `lab/images/` are pushed to `local-registry:5000`, then consumed by the gateway through the `docker://local-registry:5000/...` flow.

The gateway talks to the lab Docker daemon through `DOCKER_HOST=tcp://victim-docker:2375`. It does not mount `/var/run/docker.sock` from the real host. The local registry keeps all challenge images inside the lab boundary, and the impact remains limited to the disposable Docker-in-Docker daemon.

Scenario Evidence

Vulnerable Scenario

The vulnerable scenario uses Docker MCP Gateway `v0.42.1`, which is inside the affected version range. The challenge image carries crafted OCI metadata. When the gateway processes that image, runtime-shaping metadata reaches the container launch path. The gateway logs show the injected runtime arguments, and the proof tool writes a marker inside the fake victim host exposed by `victim-docker`.

The scenario evidence focuses on:

- image label content;
- gateway logs showing image processing;
- Docker events from the isolated victim daemon;
- gateway logs showing the generated Docker runtime arguments;
- proof that the target is the fake victim host, not the real host.

Fixed Scenario

The fixed scenario uses Docker MCP Gateway `v0.42.2`. In this lab, the same `docker://` challenge reference no longer follows the removed self-contained label parsing path. The label-defined proof tool is therefore not exposed by the fixed gateway, and the runtime proof file is not written.

The comparison focuses on:

- gateway logs from `v0.42.1` and `v0.42.2`;
- Docker events from both runs;
- exposure of the label-defined `lab_write_probe` tool;
- whether the runtime proof file is written;
- handling of the suspicious `docker://` metadata path.

Upstream Remediation

Version `0.42.2` changes how metadata imported from OCI labels is parsed and trusted.

The upstream patch introduces `catalog.ImportedServer`, a narrower import schema for image-label metadata. This schema keeps descriptive metadata while excluding runtime-shaping fields such as command, volumes, user, extra hosts, secrets, policy, remote settings, and tool container runtime configuration. Imported environment declarations keep names only, not values. Imported tool definitions omit container runtime configuration.

The patch also removes the older self-contained `docker://` parsing path that loaded label metadata directly into a full server definition. For catalog snapshot imports that still read OCI label metadata, the data is parsed through the restricted import schema rather than the full runtime-capable server structure. As defense in depth, Docker flag values added after `-v`, `-u`, and `--add-host` are checked; values starting with `-` are skipped to prevent leading-hyphen values from becoming additional Docker CLI options.

Defensive Guidance

Primary remediation is upgrading Docker MCP Gateway to `0.42.2` or later.

Operational controls include:

- treating MCP server images as executable software;
- allowing only trusted MCP image publishers and registries;
- pinning approved images by digest;
- avoiding Docker socket mounts in MCP server containers;
- avoiding broad host filesystem mounts;
- isolating experimental MCP workloads;
- monitoring Docker Engine events for unexpected mounts, users, command overrides, and container creation arguments.

References

- Docker / GitHub Security Advisory GHSA-r2xf-7jw5-pjg6. Primary upstream advisory for affected versions, severity, impact, and remediation.
  https://github.com/docker/mcp-gateway/security/advisories/GHSA-r2xf-7jw5-pjg6
- GitLab Advisory Database CVE-2026-55887. Independent advisory entry for package, version range, CVSS v3.1 vector, and references.
  https://advisories.gitlab.com/golang/github.com/docker/mcp-gateway/CVE-2026-55887/
- Trend Micro Zero Day Initiative ZDI-26-363. Coordinated disclosure entry with CVSS v3.0 score, timeline, and exploitation conditions.
  https://www.zerodayinitiative.com/advisories/ZDI-26-363/
- Upstream fix diff v0.42.1…v0.42.2. Source diff used to verify the remediation details.
  https://github.com/docker/mcp-gateway/compare/v0.42.1...v0.42.2
- Patched import schema. `catalog.ImportedServer` schema that restricts OCI label imports.
  https://github.com/docker/mcp-gateway/blob/v0.42.2/pkg/catalog/import_spec.go

- Patched OCI metadata parsing path. Patched code path converting imported label metadata into a restricted server definition.
  https://github.com/docker/mcp-gateway/blob/v0.42.2/pkg/workingset/workingset.go
- Patched Docker argument guard. Defense-in-depth checks for Docker flag values.
  https://github.com/docker/mcp-gateway/blob/v0.42.2/pkg/gateway/clientpool.go
- MITRE CWE-88. Weakness classification for improper neutralization of argument delimiters.
  https://cwe.mitre.org/data/definitions/88.html